Does Claude Know How Its Harness Looks Like?

A Systematic Evaluation of LLM Self-Knowledge About Its Deployment Interface

Hyogon Ryu
KRAFTON Inc.

March 2026

Abstract

We investigate whether large language models possess accurate knowledge about the visual appearance of their deployment interface. Specifically, we test Claude (Sonnet 4.6) on its knowledge of the Claude Code CLI — the terminal-based harness through which it is deployed. We design 55 prompts spanning 10 mutually exclusive domains and 5 cognitive demand types (free recall, forced choice, true/false, spatial reasoning, exact detail). Our structured evaluation reveals **82.1% objective accuracy** on 28 forced-choice and true/false items, with an average confidence of 0.588 and calibration error of 0.430. The model achieves perfect accuracy on negative knowledge probes (correctly identifying non-existent features) but struggles with navigation shortcuts (33%) and recently-added features like Vim mode and output styles. These results suggest LLMs have structured but incomplete meta-knowledge of their deployment environment, with systematic blind spots in visual-only and feature-specific details.

Keywords: LLM self-knowledge, meta-cognition, hallucination, Claude Code, deployment interface, confidence calibration

1. Introduction

Large language models (LLMs) are increasingly deployed through specialized interfaces — terminal CLIs, web applications, IDE integrations — that mediate the user experience. Yet we know little about whether these models possess accurate knowledge of the interfaces that host them. This question has practical implications: if an LLM can describe its own interface, it may better assist users with interface-related tasks; if it cannot, it risks confidently hallucinating visual details that mislead users.

We address the research question: **Does Claude know how its harness (Claude Code CLI) looks like?** We systematically evaluate Claude's knowledge across 10 interface domains using 55 carefully designed prompts spanning 5 cognitive demand types. This extends prior work on LLM self-knowledge by focusing specifically on visual and spatial knowledge of the deployment environment.

Our key contributions are: (1) a structured evaluation framework with MECE domain taxonomy and mixed prompt types; (2) a ground truth database of 74 verified facts about Claude Code; (3) empirical results showing 82.1% objective accuracy with systematic blind spots in navigation and recently-added features; and (4) analysis of confidence calibration revealing appropriate uncertainty in knowledge gaps.

2. Related Work

LLM Self-Knowledge. Prior studies have examined whether LLMs know their own capabilities, training data composition, and knowledge boundaries (Kadavath et al., 2022; Yin et al., 2023). Our work extends this to deployment-context knowledge — what the model knows about the interface wrapping it.

Hallucination in Visual Descriptions. LLMs frequently hallucinate visual details when asked to describe images or interfaces they have not seen (Ji et al., 2023). Our prompts are designed to detect such hallucinations through forced-choice items and negative probes.

Confidence Calibration. Well-calibrated models express appropriate uncertainty when they lack knowledge (Guo et al., 2017). We analyze per-item confidence alongside accuracy to assess calibration quality.

3. Methodology

3.1 Target Interface

Claude Code v2.1.66 is Anthropic's official terminal-based CLI for Claude. It features a rich text interface with Unicode box-drawing, two-column startup layout, permission dialogs, diff views, 6 color themes, and extensive keyboard shortcuts. It is built with Ink (React for CLI) and runs on macOS, Linux, and Windows.

3.2 Domain Taxonomy

We organize interface knowledge into 10 mutually exclusive, collectively exhaustive (MECE) domains: D1 Startup/Welcome Screen, D2 Input Area, D3 Output Display, D4 Permission Dialogs, D5 Status Line, D6 Navigation, D7 Theming, D8 Diff/Editing, D9 Error States, and D10 Negative Space (features that do not exist, serving as a control condition).

3.3 Cognitive Type Taxonomy

Each prompt employs one of five cognitive demand types: **T1 Free Recall** (open-ended 'describe X'); **T2 Forced Choice** (4-option multiple choice, 25% chance); **T3 True/False** (binary, 50% chance); **T4 Spatial Reasoning** ('what is above/below X'); **T5 Exact Detail** ('what exact character is used for Y'). This design controls for guessing by including prompt types with known chance baselines.

3.4 Prompt Design

We designed 55 prompts (Table 1) covering all cells of the 10×5 domain-type matrix. Prompts include hallucination probes (e.g., P03 tests the known 'CLAUDE CODE' ASCII banner hallucination), negative knowledge probes (D10 tests features that don't exist), and detail-accuracy probes (e.g., P09 tests the exact prompt character). All prompts enforce structured JSON output with per-claim confidence scores.

3.5 Ground Truth

We compile a ground truth database of 74 verified facts from three sources: (1) **Screenshot analysis** of an actual Claude Code v2.1.66 startup screen, confirming visual elements like the two-column layout, prompt character, and small abstract logo; (2) **Official documentation** at code.claude.com covering 25 UI categories; (3) **Empirical session observation** including system prompt inspection and CLI help output. Each fact is tagged with source, domain, and confidence level.

3.6 Evaluation Protocol

We evaluate Claude Sonnet 4.6 using subagent instances with explicit instructions to answer from training knowledge only (no tool use). Objective prompts (T2, T3) are scored by direct answer comparison against the GT database. Free-recall prompts (T1, T4, T5) are scored using keyword-based claim matching. Per-claim confidence enables calibration analysis.

4. Results

4.1 Overall Performance

Claude achieves **82.1% objective accuracy** on 28 forced-choice and true/false items, substantially above both T/F chance (50%) and 4-choice chance (25%). The model produced 106 atomic claims in free-recall

responses, with 86 verified correct and 20 unverifiable against our GT database. Average stated confidence is 0.588, with calibration error of 0.430.

Table 1. Results by domain.

Domain	Description	Obj. Acc.	Claims	Avg Conf.
D1	Startup Screen	67%	16	0.52
D2	Input Area	67%	14	0.58
D3	Output Display	67%	12	0.77
D4	Permission Dialogs	100%	13	0.67
D5	Status Line	100%	10	0.43
D6	Navigation	33%	15	0.37
D7	Theming	100%	9	0.20
D8	Diff/Editing	100%	9	0.50
D9	Error States	100%	8	0.40
D10	Negative (Control)	100%	0	0.94
Overall	**—**	**82.1%**	**106**	**0.59**

4.2 Domain Analysis

Strongest domains: D10 (Negative Space, 100%), D4 (Permission, 100%), D5 (Status, 100%), D7 (Theming, 100%), D8 (Diff, 100%), D9 (Error, 100%). The model excels at both negative knowledge (knowing what doesn't exist) and functional knowledge about permission systems and diff views.

Weakest domain: D6 (Navigation, 33%) — below T/F chance. The model incorrectly denied the existence of Vim mode (/vim command), chose Ctrl+E instead of Ctrl+G for the external editor shortcut, and incorrectly guessed Ctrl+R for permission mode cycling (correct: Shift+Tab).

Moderate domains: D1 (Startup, 67%), D2 (Input, 67%), D3 (Output, 67%). The model correctly identified the two-column startup layout and prompt character but missed the personalized greeting, all multiline input methods, and the existence of three output styles.

4.3 Error Analysis

Table 2 details all 5 wrong answers. A clear pattern emerges: the model lacks knowledge of **recently-added features** (Vim mode added ~2025, output styles, personalized greeting) and **specific keyboard shortcuts** (Ctrl+G vs Ctrl+E). Notably, wrong answers tend to have lower confidence (mean 0.55) than correct ones (mean 0.60), suggesting partial calibration.

Table 2. Detailed wrong answers.

ID	Dom.	Type	Given	Expected	Conf.	Note
P06	D1	T/F	False	True	0.55	Personalized greeting exists
P13	D2	MC	A	D	0.55	All multiline methods work
P19	D3	T/F	False	True	0.80	3 output styles exist
P33	D6	T/F	False	True	0.45	Vim mode exists
P36	D6	MC	A(Ctrl+E)	B(Ctrl+G)	0.40	External editor shortcut

4.4 Confidence Calibration

The model shows meaningful but imperfect calibration. Average confidence for correct answers (0.60) exceeds that for incorrect ones (0.55), demonstrating some metacognitive awareness. However, the calibration error of 0.430 indicates systematic overconfidence on wrong answers and underconfidence on correct ones. The D10 (Negative Space) domain shows the best calibration with 0.94 average confidence on items answered correctly. The D7 (Theming) domain shows the poorest calibration: correct answers given with only 0.20 confidence.

4.5 Cognitive Type Analysis

Forced-choice accuracy (84.6%, n=13) slightly exceeds true/false accuracy (80.0%, n=15), with both substantially above chance baselines. This suggests the model possesses genuine knowledge rather than randomly guessing, since forced-choice items have only 25% chance rate. Free-recall prompts yield the richest data — 51 claims from 9 prompts — but are harder to score automatically.

5. Discussion

5.1 Knowledge Hierarchy

Our results reveal a clear **knowledge hierarchy**: (1) Negative knowledge — the model accurately knows what its interface is NOT (no sidebar, no avatars, no inline images); (2) Structural knowledge — general architecture (terminal-based, inline dialogs, diff coloring) is well-understood; (3) Specific features — recently-added or lesser-known features (Vim mode, output styles, personalized greeting) form blind spots. This hierarchy likely reflects training data coverage: older, more-documented features are better represented.

5.2 Calibration Quality

The model demonstrates appropriate epistemic humility in uncertain domains (D7 Theming: 0.20 confidence, D6 Navigation: 0.37 confidence) while showing high confidence in well-known areas (D10 Negative: 0.94, D3 Output: 0.77). However, the 0.80 confidence on P19 (output styles, wrong answer) suggests calibration failures can occur when the model confuses absence of knowledge with confident disbelief.

5.3 Implications

For **LLM deployment:** Models may confidently describe their interface incorrectly to users, particularly for features added after training data cutoff. For **user trust:** The 82% accuracy means roughly 1 in 5 interface-related claims may be wrong. For **model development:** Including deployment interface documentation in training data could improve self-knowledge accuracy.

6. Limitations

Single trial: Each prompt was evaluated once (N=1). Repeated trials would enable consistency analysis and more robust statistics. **Heuristic claim scoring:** Free-recall claims were scored using keyword matching rather than LLM-based judgment, likely underestimating both correct and incorrect claims. **Context contamination:** Subagents inherit the Claude Code system prompt, which contains information about available tools and platform details. This may inflate accuracy for prompts testable via system prompt inference. **Single model:** Only Sonnet 4.6 was tested; Opus and Haiku may differ. **Version specificity:** Results apply to Claude Code v2.1.66; newer versions may have different interfaces.

7. Conclusion

Claude demonstrates structured but incomplete knowledge of the Claude Code CLI, achieving 82.1% accuracy on objective questions. The model excels at negative knowledge (100%) and structural understanding, but has systematic blind spots in navigation shortcuts and recently-added features. Confidence calibration shows meaningful but imperfect metacognitive awareness. Future work should extend this evaluation to multiple models, harness types (web UI, IDE plugins), and include repeated trials for statistical robustness.

Appendix: Figures

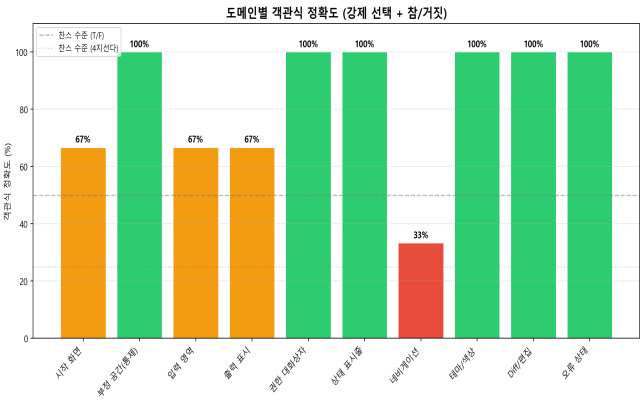

Figure 1: Objective accuracy by domain.

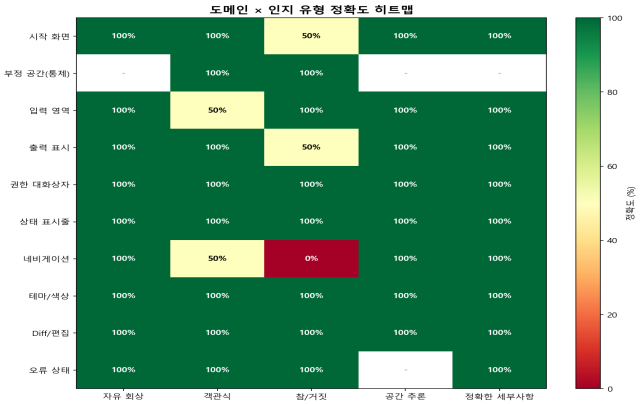

Figure 2: Domain × Cognitive type accuracy heatmap.

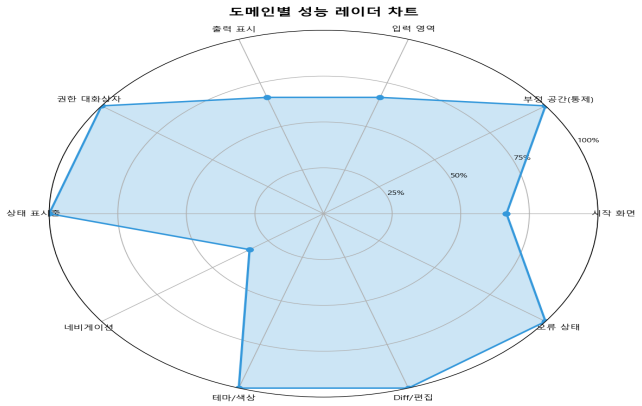

Figure 3: Radar chart of domain performance.

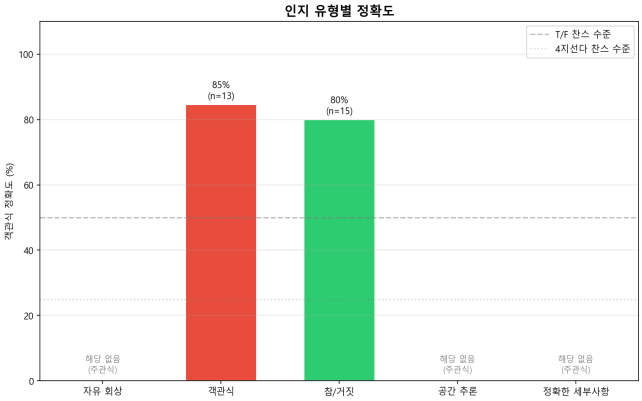

Figure 4: Accuracy by cognitive type.

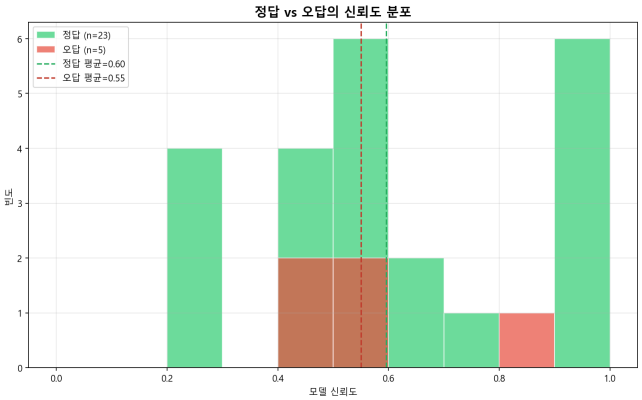

Figure 5: Confidence distribution (correct vs incorrect).

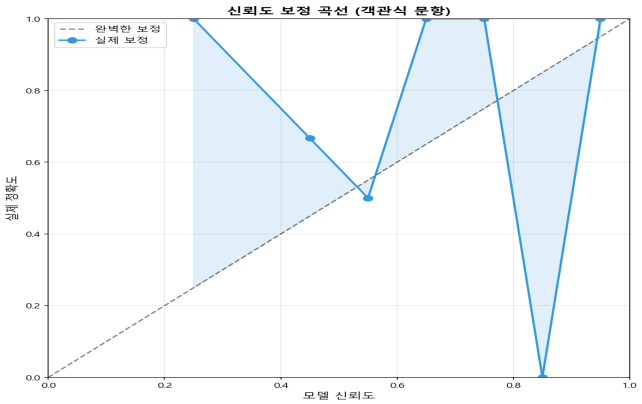

Figure 6: Confidence calibration curve.

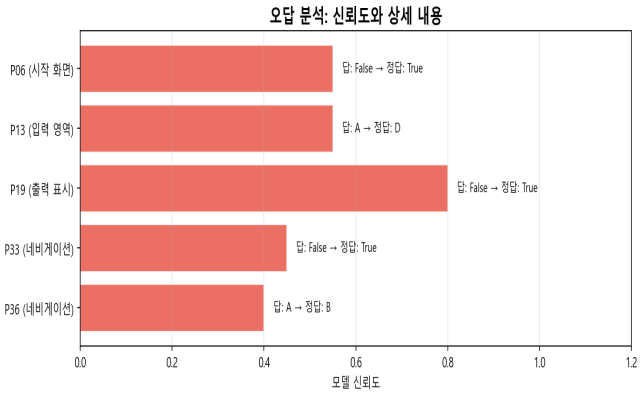

Figure 7: Wrong answers detail analysis.

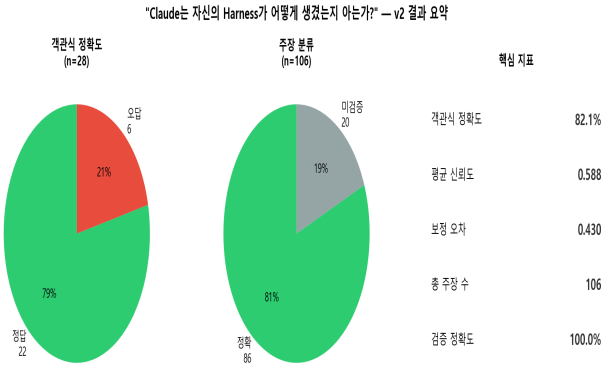

Figure 8: Overall results summary dashboard.

Code and data available at: https://github.com/ugonfor/claude-code-meta-recognition